

Types

mod types

FUNCTION	DESCRIPTION
<code>interrupt</code>	Interrupt the graph with a resumable exception from within a node.

attr All module-attribute

```
All = Literal['*']
```



Special value to indicate that graph should interrupt on all nodes.

attr StreamMode module-attribute

```
StreamMode = Literal[  
    "values", "updates", "checkpoints", "tasks", "debug", "messages", "custom"  
]
```



How the stream method should emit outputs.

- `"values"`: Emit **all** values in the state after each step, including interrupts. When used with functional API, values are emitted once at the end of the workflow.
- `"updates"`: Emit only the node or task names and updates returned by the nodes or tasks after each step. If multiple updates are made in the same step (e.g. multiple nodes are run) then those updates are emitted separately.
- `"custom"`: Emit custom data using from inside nodes or tasks using `StreamWriter`.
- `"messages"`: Emit LLM messages token-by-token together with metadata for any LLM invocations inside nodes or tasks.
- `"checkpoints"`: Emit an event when a checkpoint is created, in the same format as returned by `get_state()`.
- `"tasks"`: Emit events when tasks start and finish, including their results and errors.

- "debug": Emit "checkpoints" and "tasks" events for debugging purposes.

attr StreamWriter module-attribute

```
StreamWriter = Callable[[Any], None]
```

`Callable` that accepts a single argument and writes it to the output stream. Always injected into nodes if requested as a keyword argument, but it's a no-op when not using `stream_mode="custom"`.

class RetryPolicy

Bases: `NamedTuple`

Configuration for retrying nodes.

 **Added in version 0.2.24**

attr initial_interval class-attribute instance-attribute

```
initial_interval: float = 0.5
```

Amount of time that must elapse before the first retry occurs. In seconds.

attr backoff_factor class-attribute instance-attribute

```
backoff_factor: float = 2.0
```

Multiplier by which the interval increases after each retry.

attr max_interval class-attribute instance-attribute

```
max_interval: float = 128.0
```

Maximum amount of time that may elapse between retries. In seconds.

attr max_attempts class-attribute instance-attribute

```
max_attempts: int = 3
```

Maximum number of attempts to make before giving up, including the first.

attr jitter class-attribute instance-attribute

```
jitter: bool = True
```

Whether to add random jitter to the interval between retries.

attr retry_on class-attribute instance-attribute

```
retry_on: type[Exception] | Sequence[type[Exception]] | Callable[[Exception], bool] = (
    default_retry_on
)
```

List of exception classes that should trigger a retry, or a callable that returns `True` for exceptions that should trigger a retry.

class CachePolicy dataclass

Bases: `Generic[KeyFuncT]`

Configuration for caching nodes.

attr key_func class-attribute instance-attribute

```
key_func: KeyFuncT = default_cache_key
```

Function to generate a cache key from the node's input. Defaults to hashing the input with pickle.

attr ttl class-attribute instance-attribute

```
ttl: int | None = None
```

Time to live for the cache entry in seconds. If `None`, the entry never expires.

class Interrupt dataclass

Information about an interrupt that occurred in a node.

 **Added in version 0.2.24**

**Changed in version v0.4.0**

- `interrupt_id` was introduced as a property

**Changed in version v0.6.0**

The following attributes have been removed:

- `ns`
- `when`
- `resumable`
- `interrupt_id`, deprecated in favor of `id`

attr `id` instance-attribute

```
id: str
```



The ID of the interrupt. Can be used to resume the interrupt directly.

attr `value` instance-attribute

```
value: Any = value
```



The value associated with the interrupt.

class `PregelTask`

Bases: `NamedTuple`

A Pregel task.

class `StateSnapshot`

Bases: `NamedTuple`

Snapshot of the state of the graph at the beginning of a step.

attr `values` instance-attribute



```
values: dict[str, Any] | Any
```

Current values of channels.

attr **next** instance-attribute

```
next: tuple[str, ...]
```

The name of the node to execute in each task for this step.

attr **config** instance-attribute

```
config: RunnableConfig
```

Config used to fetch this snapshot.

attr **metadata** instance-attribute

```
metadata: CheckpointMetadata | None
```

Metadata associated with this snapshot.

attr **created_at** instance-attribute

```
created_at: str | None
```

Timestamp of snapshot creation.

attr **parent_config** instance-attribute

```
parent_config: RunnableConfig | None
```

Config used to fetch the parent snapshot, if any.

attr **tasks** instance-attribute

```
tasks: tuple[PregelTask, ...]
```

Tasks to execute in this step. If already attempted, may contain an error.

attr **interrupts** instance-attribute

```
interrupts: tuple[Interrupt, ...]
```

Interrupts that occurred in this step that are pending resolution.

`class` Send

A message or packet to send to a specific node in the graph.

The `Send` class is used within a `StateGraph`'s conditional edges to dynamically invoke a node with a custom state at the next step.

Importantly, the sent state can differ from the core graph's state, allowing for flexible and dynamic workflow management.

One such example is a "map-reduce" workflow where your graph invokes the same node multiple times in parallel with different states, before aggregating the results back into the main graph's state.

ATTRIBUTE	DESCRIPTION
<code>node</code>	The name of the target node to send the message to. TYPE: <code>str</code>
<code>arg</code>	The state or message to send to the target node. TYPE: <code>Any</code>

Example

```
from typing import Annotated
from langgraph.types import Send
from langgraph.graph import END, START
from langgraph.graph import StateGraph
import operator

class OverallState(TypedDict):
    subjects: list[str]
    jokes: Annotated[list[str], operator.add]

def continue_to_jokes(state: OverallState):
    return [Send("generate_joke", {"subject": s}) for s in state["subjects"]]

builder = StateGraph(OverallState)
builder.add_node("generate_joke", lambda state: {"jokes": [f"Joke about {state['subject']}"]})
builder.add_conditional_edges(START, continue_to_jokes)
builder.add_edge("generate_joke", END)
graph = builder.compile()

# Invoking with two subjects results in a generated joke for each
graph.invoke({"subjects": ["cats", "dogs"]})
# {'subjects': ['cats', 'dogs'], 'jokes': ['Joke about cats', 'Joke about dogs']}
```

METHOD

DESCRIPTION

`__init__`

Initialize a new instance of the `Send` class.

meth `__init__`

```
__init__(node: str, arg: Any) -> None
```

Initialize a new instance of the `Send` class.

PARAMETER

DESCRIPTION

`node`

The name of the target node to send the message to.

TYPE: `str`

`arg`

The state or message to send to the target node.

PARAMETER	DESCRIPTION
	TYPE: <code>Any</code>

class `Command` `dataclass`

Bases: `Generic[N]`, `ToolOutputMixin`

One or more commands to update the graph's state and send messages to nodes.

PARAMETER	DESCRIPTION
<code>graph</code>	Graph to send the command to. Supported values are: <code>None</code> : the current graph <code>Command.PARENT</code> : closest parent graph TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>
<code>update</code>	Update to apply to the graph's state. TYPE: <code>Any</code> <code>None</code> DEFAULT: <code>None</code>
<code>resume</code>	Value to resume execution with. To be used together with <code>interrupt()</code> . Can be one of the following: - Mapping of interrupt ids to resume values - A single value with which to resume the next interrupt TYPE: <code>dict[str, Any]</code> <code>Any</code> <code>None</code> DEFAULT: <code>None</code>
<code>goto</code>	Can be one of the following: - Name of the node to navigate to next (any node that belongs to the specified <code>graph</code>) - Sequence of node names to navigate to next <code>Send</code> object (to execute a node with the input provided) - Sequence of <code>Send</code> objects TYPE: <code>Send</code> <code>Sequence[Send N]</code> <code>N</code> DEFAULT: <code>()</code>

class `Overwrite` `dataclass`

Bypass a reducer and write the wrapped value directly to a `BinaryOperatorAggregate` channel.

Receiving multiple `Overwrite` values for the same channel in a single super-step will raise an `InvalidUpdateError`.

Example

```
from typing import Annotated
import operator
from langgraph.graph import StateGraph
from langgraph.types import Overwrite

class State(TypedDict):
    messages: Annotated[list, operator.add]

def node_a(state: TypedDict):
    # Normal update: uses the reducer (operator.add)
    return {"messages": ["a"]}

def node_b(state: State):
    # Overwrite: bypasses the reducer and replaces the entire value
    return {"messages": Overwrite(value=["b"])}

builder = StateGraph(State)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.set_entry_point("node_a")
builder.add_edge("node_a", "node_b")
graph = builder.compile()

# Without Overwrite in node_b, messages would be ["START", "a", "b"]
# With Overwrite, messages is just ["b"]
result = graph.invoke({"messages": ["START"]})
assert result == {"messages": ["b"]}
```

attr `value` `instance-attribute`

`value: Any`

The value to write directly to the channel, bypassing any reducer.

func `interrupt`

```
interrupt(value: Any) -> Any
```



Interrupt the graph with a resumable exception from within a node.

The `interrupt` function enables human-in-the-loop workflows by pausing graph execution and surfacing a value to the client. This value can communicate context or request input required to resume execution.

In a given node, the first invocation of this function raises a `GraphInterrupt` exception, halting execution. The provided `value` is included with the exception and sent to the client executing the graph.

A client resuming the graph must use the `Command` primitive to specify a value for the interrupt and continue execution. The graph resumes from the start of the node, **re-executing** all logic.

If a node contains multiple `interrupt` calls, LangGraph matches resume values to interrupts based on their order in the node. This list of resume values is scoped to the specific task executing the node and is not shared across tasks.

To use an `interrupt`, you must enable a checkpointer, as the feature relies on persisting the graph state.

Example

```
import uuid
from typing import Optional
from typing_extensions import TypedDict

from langgraph.checkpoint.memory import InMemorySaver
from langgraph.constants import START
from langgraph.graph import StateGraph
from langgraph.types import interrupt, Command

class State(TypedDict):
    """The graph state."""

    foo: str
    human_value: Optional[str]
    """Human value will be updated using an interrupt."""

def node(state: State):
    answer = interrupt(
        # This value will be sent to the client
        # as part of the interrupt information.
        "what is your age?"
    )
    print(f"> Received an input from the interrupt: {answer}")
    return {"human_value": answer}

builder = StateGraph(State)
builder.add_node("node", node)
builder.add_edge(START, "node")

# A checkpoint must be enabled for interrupts to work!
checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {
    "configurable": {
        "thread_id": uuid.uuid4(),
    }
}

for chunk in graph.stream({"foo": "abc"}, config):
    print(chunk)

# > {'__interrupt__': (Interrupt(value='what is your age?',
id='45fda8478b2ef754419799e10992af06'),)}

command = Command(resume="some input from a human!!!")

for chunk in graph.stream(Command(resume="some input from a human!!!"), config):
    print(chunk)
```

```
# > Received an input from the interrupt: some input from a human!!!  
# > {'node': {'human_value': 'some input from a human!!!'}}
```

PARAMETER	DESCRIPTION
<code>value</code>	The value to surface to the client when the graph is interrupted. TYPE: <code>Any</code>

RETURNS	DESCRIPTION
<code>Any</code>	On subsequent invocations within the same node (same task to be precise), returns the value provided during the first invocation TYPE: <code>Any</code>

RAISES	DESCRIPTION
<code>GraphInterrupt</code>	On the first invocation within the node, halts execution and surfaces the provided value to the client.